

CSCE 669 COMPUTATIONAL OPTIMIZATION · MODULE 11 · LECTURE NOTES

Combinatorial Optimisation

The problems with special structure.

MODULE OUTCOMES

- Explain network flow and its integrality.
- Explain matching and its duality.
- Explain matroids and why greedy works on them.
- Explain submodularity and the greedy guarantee.
- Recognise exploitable structure in a new problem.

1 Network flow

MAX-FLOW MIN-CUT IS LP DUALITY MADE CONCRETE

The maximum flow from a source to a sink equals the minimum total capacity of any cut separating them. Stated this way it looks like a combinatorial coincidence.

It is strong duality. Write the maximum flow problem as a linear program; its dual is the minimum cut problem, and Module 04 §1's strong duality for linear programs gives the equality — **the famous theorem is an instance of a general one.**

And the constraint matrix of a flow problem is totally unimodular (Module 03 §3), so every vertex of the polyhedron is integral and the LP optimum is automatically a valid integer flow.

So the problem is polynomial for a structural reason, not because a sufficiently clever algorithm was eventually found. Ford–Fulkerson, Dinic, and push-relabel exploit the structure; they do not create it — and that distinction is what this module is about, because it tells you where to look when facing a new problem.

Problem	Reduction	Note
Bipartite matching	Unit capacities: source to every left vertex, every right vertex to sink, edges in between.	An integral maximum flow is a maximum matching, and integrality is guaranteed.
Vertex-disjoint paths	Split each vertex into an in-node and an out-node joined by a capacity-1 edge.	Menger's theorem falls out as the min-cut.
Project selection	Minimum cut on a graph of projects and their prerequisite costs.	Maximum closure — a surprising and very useful reduction for selection problems with dependencies.

Problem	Reduction	Note
Image segmentation	Minimum cut with data terms on vertices and smoothness terms on edges.	Graph cuts — the standard method before deep learning, and still used (CSCE 748). Exact for binary labelling.
Scheduling with release times and deadlines	Flow over a time-expanded graph.	Common in practice, and frequently missed.
Baseball elimination	Flow feasibility.	The classic surprising reduction — determining whether a team can still win is a max-flow computation.

2 Matroids

A MATROID is a ground set E with a family of INDEPENDENT subsets satisfying:

1. the empty set is independent
2. subsets of independent sets are independent (hereditary)
3. EXCHANGE: if $|A| < |B|$ and both are independent, some element of B can be added to A keeping it independent

THEOREM: the greedy algorithm -- sort by weight, add greedily whenever independence is preserved -- finds the MAXIMUM WEIGHT independent set EXACTLY, for ANY matroid.

CONVERSE: if greedy is optimal for every weighting, the structure IS a matroid.

SO MATROIDS ARE EXACTLY THE STRUCTURES WHERE GREEDY WORKS.

EXAMPLES

graphic matroid	forests in a graph -> KRUSKAL'S MST
linear matroid	linearly independent sets of columns
partition matroid	at most k_i from each group
uniform matroid	any set of size $\leq k$

That the converse holds is what makes this result satisfying. It is a *complete characterisation* of when greedy is optimal — an unusually clean answer to a question that is normally answered case by case. **So Kruskal's algorithm is not a clever trick specific to spanning trees; it is the matroid greedy algorithm applied to the graphic matroid** (CSCE 629), and recognising that explains why it works rather than merely that it does.

3 Submodularity

$$f(A \cup \{x\}) - f(A) \geq f(B \cup \{x\}) - f(B) \quad \text{for } A \subseteq B$$

Adding an element to a smaller set helps at least as much as adding it to a larger one — diminishing returns, formalised. **Maximising a monotone submodular function subject to a cardinality constraint is NP-hard, and the greedy algorithm achieves $(1 - 1/e) \approx 0.632$ of the optimum** — Nemhauser, Wolsey and Fisher, 1978. **And $1 - 1/e$ is itself optimal unless $P = NP$ (Module 10 §4), so greedy is the best polynomial-time algorithm available.** Submodularity is to discrete optimisation what convexity is to continuous optimisation — the structural property that converts an intractable problem into one with a guarantee, and the analogy is close enough to be worth carrying.

WHERE SUBMODULARITY SHOWS UP

Coverage functions. How many distinct items a chosen set covers — sensor placement, facility location, influence maximisation in networks, test-case selection. **Any 'cover as much as possible with k choices' problem.**

Entropy and mutual information of a selected set of random variables are submodular — **so experimental design, sensor selection, and active learning (CSCE 633 Module 11 §4) all inherit the guarantee**, which is a genuinely useful fact that is not widely known in machine learning.

Diversity and summarisation objectives — selecting a representative subset of documents, images, or data points, where adding a near-duplicate of something already chosen adds little.

And feature selection under certain independence assumptions. So the 0.632 guarantee covers a wide class of practical selection problems where people currently use greedy and hope — checking submodularity converts the hope into a theorem, and it is frequently a short check.

4 Recognising structure

- **Is it a flow problem in disguise?** Look for conservation, capacities, a bipartite assignment, or a two-way partition. **Check this first** — it is the highest-value pattern-match in the module (§1).
- **Is the constraint matrix totally unimodular?** Then drop the integrality constraints entirely and solve the LP (Module 03 §3) — interval structure, node-arc incidence, and consecutive-ones structure are the usual signs.
- **Is the family of feasible sets a matroid?** Then the greedy algorithm is exact (§2), and no further work is needed.
- **Is the objective monotone and submodular, under a cardinality or matroid constraint?** Then greedy is within 0.632 and that is optimal (§3).
- **Otherwise: integer programming (Module 09) with the strongest formulation you can construct. This checklist is worth running before writing any code** — each hit converts an apparently intractable problem into a solved one, and the check costs an hour.

STRUCTURE IS FOUND BY MODELLING, NOT BY SEARCHING

Whether a problem has exploitable structure frequently depends on how you formulate it rather than on the problem itself. The structure is a property of the mathematical model, and you chose the model.

The same practical situation, modelled with different decision variables, can be a network flow problem or an arbitrary integer program. Choosing 'how much flows along this arc' rather than 'which assignment is made' can be the difference between a polynomial algorithm and an exponential search.

So spend real time on the formulation before accepting that a problem is hard. 'This is NP-hard' is a statement about a formulation at least as often as about a problem, and reformulating is cheaper than any algorithmic heroics.

Which is Module 12's subject, and the reason it follows this module rather than preceding it — **you need the catalogue of exploitable structures before you can usefully aim a formulation at one.**

Where to see this taught differently

Original material only goes so far — hearing the same idea in another voice is how it sticks. These are free, and they cover this module specifically.

Resource	What it covers for this module
Williamson & Shmoys, and Vazirani — the combinatorial chapters (free)	Flow, matching, and the primal-dual method, with the duality connections of §1 made explicit.
Schrijver — Combinatorial Optimization: Polyhedra and Efficiency	The comprehensive reference for §1 and §2. The author's shorter lecture notes are free and are the right entry point.
Oxley — Matroid Theory; and Edmonds' greedy papers	The §2 characterisation and its converse. Edmonds' original papers are short and readable.
Krause & Golovin — Submodular Function Maximization (free survey)	The §3 material , with the applications of the callout and the lazy-greedy speedup that makes it practical.

Exercises

Do these in code, not on paper, unless the exercise says otherwise. The module is not finished until they run.

- 1 Formulate maximum flow as a linear program** and derive its dual. Confirm the dual is minimum cut.
- Solve the LP and verify the optimum is integral without being constrained to be.
- 3 Reduce bipartite matching to flow** and solve an instance both ways.
- 4 Implement graph-cut image segmentation** on a small image and connect it to CSCE 748.

- 5 **Verify the matroid axioms** for forests in a graph.
- 6 **Implement matroid greedy** generically and instantiate it as Kruskal's algorithm.
- 7 **Construct a structure that is not a matroid** and exhibit a weighting where greedy fails.
- 8 **Verify submodularity** of a coverage function by checking the diminishing-returns inequality.
- 9 **Run greedy on a submodular maximisation** and compare against the exact optimum on small instances. Confirm the ratio exceeds 0.632.
- 10 Take a problem from your own work and run §4's checklist on it.

Self-check

Answer these in writing, from memory, without looking back at the notes. If you cannot, you have read this module but not yet learned it — that is useful information, not a failure.

- 1 Why is max-flow min-cut a duality result, and where does integrality come from?
- 2 Name six problems that reduce to flow.
- 3 State the matroid axioms and the greedy theorem, including its converse.
- 4 Why is Kruskal's algorithm not a special trick?
- 5 Define submodularity and state the greedy guarantee and its optimality.
- 6 Give four places submodularity appears.
- 7 Give the five-step structure checklist.
- 8 Why is structure a property of the formulation?